

Anki Decks with Orgmode

Rohit Goswami · 2020-10-27 · workflow · projects · tools · emacs · orgmode

Setting up unicode math and `orgmode` for painless Anki deck building

Background

A recent [Hacker News post](#) reminded me of [Anki](#), and that brought back memories of my Anki `orgmode` setup. I thought I'd re-create and immortalize it.

The standard way of working with Anki, is with a pretty awkward GUI. There are changes to be made here, which make life a little easier, including the setup of custom cards, but the inherent concerns of the WYSIWYG editor are basically insurmountable.

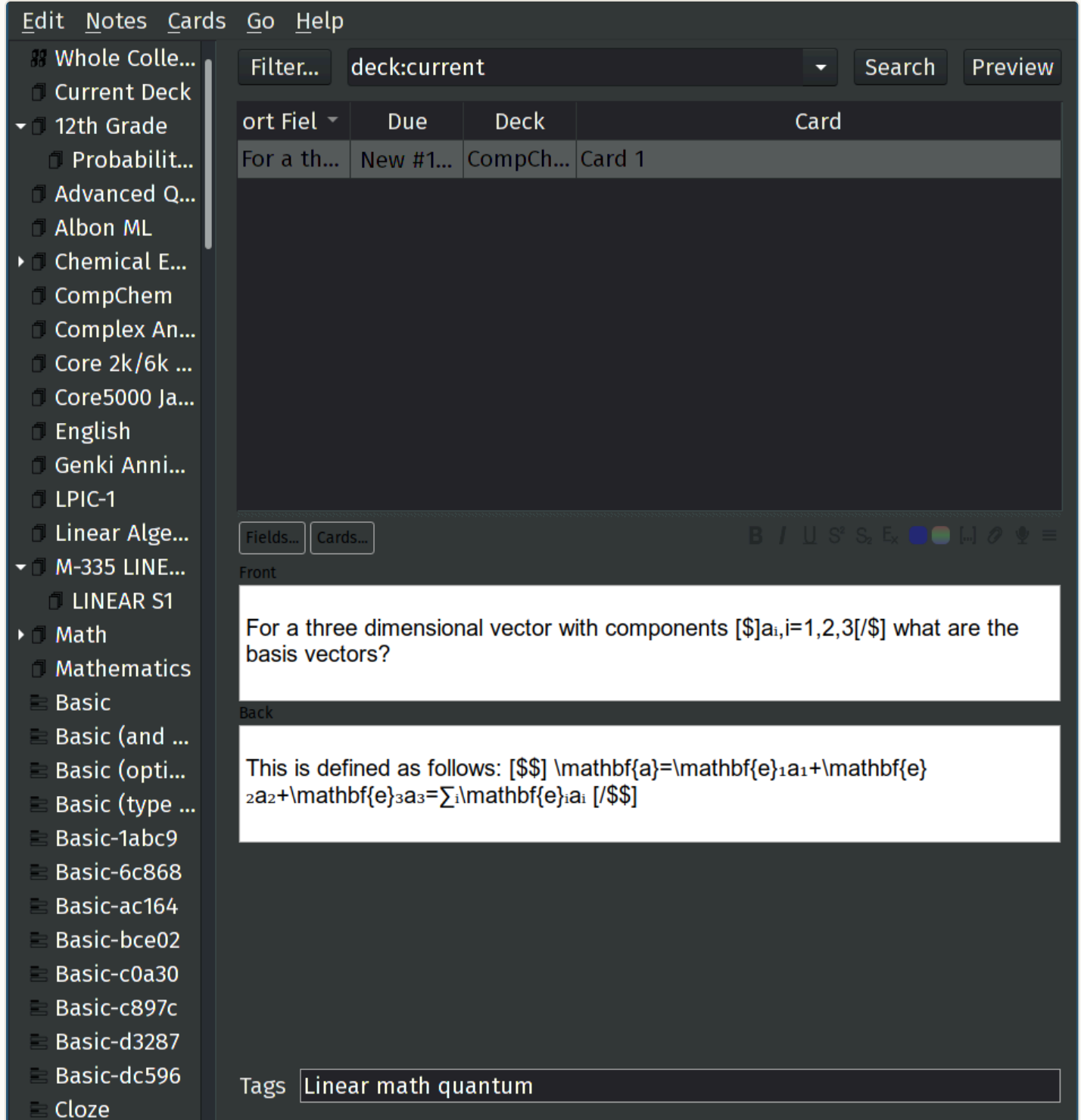


Figure 1: Anki GUI

The goal is to get this a better workflow than manual editing of Anki decks. `orgmode` is perfect for making cards, especially in the larger context of using it for storing images and rich `pdfs`.

```

5 * About
6 This is a set of cards and notes to be augmented by some Anki based spaced repetition
  as effected by anki-editor along with the beautiful Stylus Labs Write for proofs. The
  Setup is a combination of my multi-monitor setup with constant touchscreen inputs.
7
8 * Basis Vectors :math:quantum:linear:
9 :properties:
10 :anki_deck: CompChem
11 :anki_note_type: LaTeX
12 :ANKI_NOTE_ID: 1603755931922
13 :end:
14 * Front
15 For a three dimensional vector with components  $a_i, i=1,2,3$  what are the basis vectors?
16 * Back
17 This is defined as follows:
18 $$
19 \mathbf{a} = \mathbf{e}_1 a_1 + \mathbf{e}_2 a_2 + \mathbf{e}_3 a_3 = \sum_i \mathbf{e}_i a_i
20 $$
21
22
23

```

870 courseworkUoI/fall2020/EFN115F/revCards/reviewCompChem.org 6:172 Bot LF UTF-8 Org (+3)

Figure 2: A pleasant way to make anki decks

Methodology

To accomplish this, we basically need to have the following:

anki-editor

This `emacs` plugin will facilitate the conversion from our `orgmode` files to the Anki markup

anki-connect

We need a server of sorts set up to allow us to push pull and get errors from the running Anki server, this is an Anki plugin

LaTeX process editor

It wouldn't be much better than manually making cards in Anki if we couldn't leverage `unicode` characters, so we need to modify the internal Anki build process for TeX

Anki Editor

As with all `emacs` related setup snippets on this site, these should be modified and adapted as needed, especially for those not using [doom-emacs](#).

```

(use-package anki-editor
  :after org-noter
  :config
  ; I like making decks
  (setq anki-editor-create-decks 't))

```

Also, my [full configuration](#) has additional non-essential quality of life keybindings amongst other things.

Anki Connect

`CTRL+Shift+A` will bring up the addon settings, and Anki has to be restarted after installing the addons. [Anki Connect](#) itself does not need any further configuration, though the `readme` is very thorough.

TeX Setup

The [LaTeX process editor](#) can be set in two stages, wherein we will first ensure that we can use `xelatex` and that we can generate an `svg`.

```
{
  "svgCommands": [
    [
      "xelatex",
      "--no-pdf",
      "-interaction=nonstopmode",
      "tmp.tex"
    ],
    [
      "dvisvgm",
      "--no-fonts",
      "-Z",
      "2",
      "tmp.xdv",
      "-o",
      "tmp.svg"
    ]
  ]
}
```

The `png` settings can be modified in a similar manner if required, but it is better to generate `svg` files, which will set up in the cosmetics section. Note that we pass `--no-pdf` to get the `xdv` file which has replaced `dvi` files for `xelatex`.

Cosmetics

The final aspect of this is to be configured with the GUI. The easiest option is to clone the Basic card type and customize that. `CTRL+Shift+N` should bring up the card editor. The relevant styles are^[^fn:1] (from the `Cards` option):

```
.card {
  font-family: Literata;
  font-size: 26px;
  text-align: center;
  color: black;
  background-color: white;
}
img {
  max-height: 1000px;
  height: auto;
  width: auto;
}
img[src*="latex"] {
  vertical-align: middle;
}
```

Now we need setup our TeX headers as well, and enable the `Create scalable images with dvisvgm` option. The header needs to have (minimally):

```

\documentclass[12pt]{article}
\special{papersize=3in,5in}
\usepackage{geometry}
\usepackage{unicode-math}
\usepackage{mathtools}
\pagestyle{empty}
\setlength{\parindent}{0in}
\begin{document}

```

While the `footer` is simply `\end{document}`. With this, we have achieved pretty formatting.

For a three dimensional vector with components $a_i, i = 1, 2, 3$ what are the basis vectors?

This is defined as follows:

$$\mathbf{a} = \mathbf{e}_1 a_1 + \mathbf{e}_2 a_2 + \mathbf{e}_3 a_3 = \sum_i \mathbf{e}_i a_i$$

Figure 3: Pretty card formatting

Font Locking

Inspired by [this post](#), we will also use [orgcss](#) to obtain some `orgmode` font-locking. We will add the following styles:

```

:not(pre) > code {
  padding: 2px 5px;
  margin: auto 1px;
  border: 1px solid #ddd;
  border-radius: 3px;
  background-clip: padding-box;
  color: #333;
  font-size: $code-size;
}

.org-src-container {
  border: 1px solid #ccc;
  box-shadow: 3px 3px 3px #eee;
  font-family: $monospace;
  font-size: $code-size;
  margin: 1em auto;
  padding: 0.1em 0.5em;
  position: relative;
}

.org-src-container > pre {
  overflow: auto;
}

.org-src-container > pre:before {
  display: block;
  position: absolute;
  background-color: #b3b3b3;
  top: 0;
  right: 0;
  padding: 0 0.5em;
  border-bottom-left-radius: 8px;
  border: 0;
  color: white;
  font-size: $code-size;
}

/* from http://demo.thi.ng/org-spec/ */

.org-src-container > pre.src-sh:before {
  content: "sh";
}

.org-src-container > pre.src-bash:before {
  content: "bash";
}

.org-src-container > pre.src-emacs-lisp:before {
  content: "Emacs Lisp";
}

.org-src-container > pre.src-R:before {
  content: "R";
}

.org-src-container > pre.src-org:before {
  content: "Org";
}

.org-src-container > pre.src-cpp:before {
  content: "C++";
}

.org-src-container > pre.src-c:before {
  content: "C";
}

.org-src-container > pre.src-html:before {
  content: "HTML";
}

.org-src-container > pre.src-js:before {
  content: "Javascript";
}

```

```
}
.org-src-container > pre.src-javascript:before {
  content: "Javascript";
}

// More languages from http://orgmode.org/worg/org-contrib/babel/languages.html

.org-src-container > pre.src-abc:before {
  content: "ABC";
}
.org-src-container > pre.src-asymptote:before {
  content: "Asymptote";
}
.org-src-container > pre.src-awk:before {
  content: "Awk";
}
.org-src-container > pre.src-C:before {
  content: "C";
}
.org-src-container > pre.src-calc:before {
  content: "Calc";
}
.org-src-container > pre.src-clojure:before {
  content: "Clojure";
}
.org-src-container > pre.src-comint:before {
  content: "comint";
}
.org-src-container > pre.src-css:before {
  content: "CSS";
}
.org-src-container > pre.src-D:before {
  content: "D";
}
.org-src-container > pre.src-ditaa:before {
  content: "Ditaa";
}
.org-src-container > pre.src-dot:before {
  content: "Dot";
}
.org-src-container > pre.src-ebnf:before {
  content: "ebnf";
}
.org-src-container > pre.src-forth:before {
  content: "Forth";
}
.org-src-container > pre.src-F90:before {
  content: "Fortran";
}
.org-src-container > pre.src-gnuplot:before {
  content: "Gnuplot";
}
.org-src-container > pre.src-haskell:before {
  content: "Haskell";
}
.org-src-container > pre.src-io:before {
  content: "Io";
}
.org-src-container > pre.src-java:before {
  content: "Java";
}
.org-src-container > pre.src-latex:before {
  content: "LaTeX";
}
.org-src-container > pre.src-ledger:before {
  content: "Ledger";
}
```

```
}
.org-src-container > pre.src-ly:before {
  content: "Lilypond";
}
.org-src-container > pre.src-lisp:before {
  content: "Lisp";
}
.org-src-container > pre.src-makefile:before {
  content: "Make";
}
.org-src-container > pre.src-matlab:before {
  content: "Matlab";
}
.org-src-container > pre.src-max:before {
  content: "Maxima";
}
.org-src-container > pre.src-mscgen:before {
  content: "Mscgen";
}
.org-src-container > pre.src-Caml:before {
  content: "Objective";
}
.org-src-container > pre.src-octave:before {
  content: "Octave";
}
.org-src-container > pre.src-org:before {
  content: "Org";
}
.org-src-container > pre.src-perl:before {
  content: "Perl";
}
.org-src-container > pre.src-picolisp:before {
  content: "Picolisp";
}
.org-src-container > pre.src-plantuml:before {
  content: "PlantUML";
}
.org-src-container > pre.src-python:before {
  content: "Python";
}
.org-src-container > pre.src-ruby:before {
  content: "Ruby";
}
.org-src-container > pre.src-sass:before {
  content: "Sass";
}
.org-src-container > pre.src-scala:before {
  content: "Scala";
}
.org-src-container > pre.src-scheme:before {
  content: "Scheme";
}
.org-src-container > pre.src-screen:before {
  content: "Screen";
}
.org-src-container > pre.src-sed:before {
  content: "Sed";
}
.org-src-container > pre.src-shell:before {
  content: "shell";
}
.org-src-container > pre.src-shen:before {
  content: "Shen";
}
.org-src-container > pre.src-sql:before {
  content: "SQL";
}
```



```
}
.org-src-container > pre.src-sqlite:before {
  content: "SQLite";
}
.org-src-container > pre.src-stan:before {
  content: "Stan";
}
.org-src-container > pre.src-vala:before {
  content: "Vala";
}
.org-src-container > pre.src-axiom:before {
  content: "Axiom";
}
.org-src-container > pre.src-browser:before {
  content: "HTML";
}
.org-src-container > pre.src-cypher:before {
  content: "Neo4j";
}
.org-src-container > pre.src-elixir:before {
  content: "Elixir";
}
.org-src-container > pre.src-request:before {
  content: "http";
}
.org-src-container > pre.src-ipython:before {
  content: "iPython";
}
.org-src-container > pre.src-kotlin:before {
  content: "Kotlin";
}
.org-src-container > pre.src-Flavored Erlang lfe:before {
  content: "Lisp";
}
.org-src-container > pre.src-mongo:before {
  content: "MongoDB";
}
.org-src-container > pre.src-prolog:before {
  content: "Prolog";
}
.org-src-container > pre.src-rec:before {
  content: "rec";
}
.org-src-container > pre.src-ML sml:before {
  content: "Standard";
}
.org-src-container > pre.src-Translate translate:before {
  content: "Google";
}
.org-src-container > pre.src-typescript:before {
  content: "Typescript";
}
.org-src-container > pre.src-rust:before {
  content: "Rust";
}
```

However, in the interests of sanity, we will leverage the [Syntax Highlighting Anki plugin](#) for managing the actual style-sheets instead of manual edits to each card type.



Figure 4: A screenshot from the plugin readme

Figure 4: A screenshot from the plugin readme

At this stage, we have a card which can gracefully handle both XeLaTeX and code in an elegant manner. An example is presented in the next section.

Usage

For the sample card^[^fn:2] shown, the markup is dead simple.

```
* Basis Vectors :math:quantum:linear:
:properties:
:anki_deck: CompChem
:anki_note_type: LaTeX
:ANKI_NOTE_ID: 1603755931922
:end:
** Front
For a three dimensional vector with components  $a_i, i=1,2,3$  what are the basis vectors?
** Back
This is defined as follows:
$$
\mathbf{a}=\mathbf{e}_1a_1+\mathbf{e}_2a_2+\mathbf{e}_3a_3=\sum_i\mathbf{e}_ia_i
$$
```

Essentially:

- Enable and load `anki-editor`
 - Add local variable section to ensure we load `anki-editor`. This is essentially via `eval: (anki-editor-mode)` in the Local variables block
- Fire up Anki
- Export at will, and continue adding more cards or non-card details to the `orgmode` file

The [Anki editor examples](#) file is excellent and the [issue tracker](#) also has a ton of information.

Code

```
* Test Code :code:python:
:properties:
:anki_deck: CodeWiki
:anki_note_type: myTex
:ANKI_NOTE_ID: 1603891864091
:end:
** Front
What is the definition of an inner product? What are some examples of a code block in Python and R?
** Back
This is essentially a *norm* with more structure. The first two properties, positive definiteness and symmetry (conjugate) defines a *norm*.
$$
\mathbf{a}=\mathbf{e}_1a_1+\mathbf{e}_2a_2+\mathbf{e}_3a_3=\sum_i\mathbf{e}_ia_i
$$

#+begin_src python
def test():
    x = [1,2,3]
    for i in x:
        print(i)
#+end_src

#+begin_src R
library("dplyr")
x = 1
#+end_src
```

FileEditToolsHelp

DecksAddBrowseStatsSync

What is the definition of an inner product? What are some examples of a code block in Python and R?

This is essentially a **norm** with more structure. The first two properties, positive definiteness and symmetry (conjugate) defines a **norm**.

$$\mathbf{a} = \mathbf{e}_1 a_1 + \mathbf{e}_2 a_2 + \mathbf{e}_3 a_3 = \sum_i \mathbf{e}_i a_i$$

Python

```
def test():  
    x = [1,2,3]  
    for i in x:  
        print(i)
```

R

```
library("dplyr")  
x = 1
```

Edit

<1mAgain<10mGood4dEasy

More ▾

Figure 5: Code card with TeX

More Content

Fundamental Haskell

An excellent example of how a multiple frontend learning repository can be, written with `org-drill` ^[^fn:3]

Anki powerups with orgmode

A post brought to my attention after I had published this, an excellent introduction with videos

Conclusions

Some final comments:

- Screenshots and other images linked are automatically synced
- The TeX is best rendered on the PC first, so run through these at-least once

A missing link in this setup is the ability to use a touch screen and stylus to write proofs or skip the TeX setup altogether, but that would require another post altogether. Additionally, all the standard bells and whistles of having an `orgmode` document can be applied, including, crucially, the ability to have long-form notes as well, a coherent approach to this can also be covered later.